

OOPs — LAST-MINUTE REVISION SHEET

Object Oriented Programming | Quick Reference for Exam

1. OOP LANGUAGES — IS IT OOP OR NOT?

Category	Languages
Pure OOP	Smalltalk, Ruby, Eiffel, Self
OOP (with procedural support)	C++, Java, Python, C#, Kotlin, Swift, Scala
NOT OOP (Procedural)	C, Pascal, Fortran, COBOL, Assembly
NOT OOP (Functional)	Haskell, Erlang, LISP (pure functional)
NOT OOP (Scripting/Mixed)	JavaScript (prototype-based, NOT pure OOP)

- **Key Rule:** Java — everything is in a class, but has primitives (int, char) → NOT purely OOP
- **Key Rule:** Python — supports OOP but also procedural → Multi-paradigm
- **C** — NOT OOP (no class/object concept); C++ — IS OOP (adds classes to C)

2. FOUR PILLARS OF OOP

Pillar	One-Line Definition	Keyword to Remember
Encapsulation	Wrapping data + methods in a single unit (class)	Data Hiding
Abstraction	Hiding implementation, showing only essentials	Interface/Abstract Class
Inheritance	Child class inherits properties of parent class	IS-A Relationship
Polymorphism	Same name, different behaviour	One interface, many forms

3. CLASS & OBJECT

Class

- **Definition:** Blueprint/template for creating objects; user-defined data type
- **Contains:** Data members (variables) + Member functions (methods)
- **Memory:** NO memory allocated for class declaration itself
- **Syntax:** `class ClassName { access_specifier: members; };`

Object

- **Definition:** Instance of a class; occupies actual memory
- **Also called:** Instance variable
- **Memory:** Allocated when object is created (instantiation)
- **Key:** One class → Multiple objects (each with own copy of data members)

Instantiation

- **Definition:** Process of creating an object from a class

- **Example:** `ClassName obj = new ClassName(); // Java`
- **Remember:** Constructor is called during instantiation

4. ACCESS SPECIFIERS

Specifier	Same Class	Subclass (same pkg)	Subclass (diff pkg)	Other Classes
private	Yes	No	No	No
default (no keyword)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

5. ENCAPSULATION

- **Definition:** Binding data (variables) and code (methods) together as a single unit
- **Achieves:** Data Hiding — prevents direct access to data from outside class
- **Implemented via:** private data members + public getter/setter methods
- **Benefit:** Data protection, modularization, flexibility
- **Example:** `private int age; // can't access directly outside class`
- **Java bean:** Perfect example of encapsulation

6. INHERITANCE

- **Definition:** Mechanism by which one class (child/derived/subclass) acquires properties and methods of another class (parent/base/superclass)
- **IS-A relationship:** Dog IS-A Animal → Dog extends Animal
- **Keyword:** `extends` (Java), `:` (C++)
- **NOT inherited:** private members, constructors, static members (accessible but not inherited)

Types of Inheritance

Type	Description	Supported In
Single	One parent → One child	C++, Java, Python
Multilevel	A→B→C (chain)	C++, Java, Python
Hierarchical	One parent → Multiple children	C++, Java, Python
Multiple	Multiple parents → One child	C++, Python ONLY (NOT Java)
Hybrid	Combination of above	C++ only (NOT Java)

- **Java does NOT support Multiple Inheritance via classes** (to avoid Diamond Problem); uses Interfaces instead
- **Diamond Problem:** Ambiguity when two parents have same method → child doesn't know which to inherit

Constructor & Destructor in Inheritance

- **Order of Constructor Call:** Parent constructor → Child constructor (always parent first)
- **Order of Destructor Call:** Child destructor → Parent destructor (reverse order)
- **Java:** `super()` calls parent constructor (must be first line in child constructor)
- **C++:** Parent constructor called automatically; use initializer list for explicit call
- **Default:** If child constructor doesn't call `super()`, compiler inserts `super()` automatically
- **Rule:** Constructor is NOT inherited but IS called during inheritance

Scenario	Constructor Order	Destructor Order
A → B (A is parent)	A() → B()	~B() → ~A()
A → B → C	A() → B() → C()	~C() → ~B() → ~A()

7. POLYMORPHISM

- **Definition:** Ability of a message/function to take multiple forms
- **Types:** Compile-time (Static) Polymorphism and Run-time (Dynamic) Polymorphism

Feature	Compile-time (Static)	Run-time (Dynamic)
Also Called	Early Binding / Static Binding	Late Binding / Dynamic Binding
Achieved By	Method Overloading, Operator Overloading	Method Overriding, Virtual Functions
Resolved At	Compile time	Runtime
Speed	Faster	Slower (due to vtable lookup)
Flexibility	Less flexible	More flexible
Java Example	Overloading	Overriding with <code>@Override</code>

Method Overloading

- **Definition:** Same method name, different parameters (in same class)
- **Differs by:** Number of parameters, types of parameters, order of parameters
- **NOT by:** Return type alone (causes compile error)
- **Keyword:** None — compiler decides based on signature

Method Overriding

- **Definition:** Child class provides specific implementation of method already in parent
- **Rule:** Same name, same return type, same parameters as parent method
- **Java keyword:** `@Override` annotation (optional but recommended)
- **Cannot override:** `final`, `static`, `private` methods

8. ABSTRACT CLASS

- **Definition:** A class declared with `abstract` keyword that CANNOT be instantiated (no object creation)
- **Contains:** Abstract methods (no body) AND/OR concrete methods (with body)
- **Purpose:** Provides partial implementation; forces subclass to implement abstract methods
- **Keyword:** `abstract class ClassName { abstract void method(); }`
- **Must:** Subclass must override ALL abstract methods, OR also be declared abstract
- **Can have:** Constructors, instance variables, concrete methods, static methods
- **Cannot:** Cannot create object (`new AbstractClass()` → compile error)

Feature	Abstract Class	Interface
Keyword	<code>abstract class</code>	<code>interface</code>
Object creation	Not possible	Not possible
Methods	Abstract + Concrete	Abstract (default: all); Java 8+ allows default/static
Variables	Any type	<code>public static final</code> only
Constructor	Yes	No
Multiple Inheritance	No (single class)	Yes (can implement multiple)
Use when	Partial implementation needed	Full abstraction needed

9. VIRTUAL FUNCTION

- **Definition:** Member function declared in base class with `virtual` keyword, redefined in derived class → enables Runtime Polymorphism
- **Language:** C++ concept (Java all non-static methods are virtual by default)
- **Syntax (C++):** `virtual void display() { ... } // in base class`
- **How it works:** Uses Virtual Table (vtable) — each class has a vtable of function pointers; resolved at runtime via vptr (virtual pointer)
- **Rule:** Called via BASE class pointer/reference → executes DERIVED class version
- **Pure Virtual Function:** `virtual void show() = 0;` → makes the class Abstract
- **Pure Virtual:** No body in base class; MUST be overridden in derived class

Type	Syntax	Class becomes
Virtual function	<code>virtual void f() { ... }</code>	Concrete (can instantiate)
Pure virtual function	<code>virtual void f() = 0;</code>	Abstract (cannot instantiate)

- **Virtual Destructor:** Use `virtual ~Base()` when deleting derived object via base pointer (avoids memory leak)
- **Java equivalent:** All non-static, non-final, non-private methods are virtual by default

10. FINAL KEYWORD (Java)

Applied to	Effect	Example
------------	--------	---------

final variable	Value cannot be changed (constant)	final int MAX = 100;
final method	Cannot be overridden by subclass	final void show() { }
final class	Cannot be extended/subclassed	final class String { }

- **C++ equivalent:** `const` for variables; `final` (C++11) for classes/virtual methods
- **Key Exam Point:** String class in Java is final — cannot be inherited
- **final ≠ static:** final = value/behaviour fixed; static = belongs to class not object

11. STATIC KEYWORD

- **Definition:** Belongs to the CLASS, not to any specific object/instance

Applied to	Meaning	Access
static variable	Shared among ALL objects of class (class variable)	ClassName.variable
static method	Can be called without creating object; cannot access non-static members	ClassName.method()
static block (Java)	Executes once when class is loaded	Automatic
static class (Java)	Only inner/nested class can be static	ClassName.InnerClass

- **Cannot use:** `this` or `super` inside static method (no object context)
- **Cannot override:** Static methods cannot be overridden (method hiding, not overriding)
- **Memory:** Allocated once in class area (not heap); shared by all objects
- **Exam Tip:** `main()` is static because JVM calls it before any object creation

12. FRIEND FUNCTION (C++ ONLY)

- **Definition:** A non-member function that has access to private and protected members of a class
- **Declared using:** `friend` keyword inside the class
- **NOT a member:** Declared inside class but defined outside; NOT called with object (`obj.friendFunc()` — WRONG)
- **Properties:**
 - NOT inherited — friendship is not inherited
 - NOT transitive — friend of a friend is NOT a friend
 - NOT mutual — A declares B as friend; B does NOT automatically get A as friend
 - Can be friend of multiple classes
 - Violates encapsulation (use sparingly)
- **Syntax:** `friend void show(ClassName obj); // inside class definition`
- **Friend Class:** All methods of friend class can access private members → `friend class AnotherClass;`
- **Java:** NO friend function concept in Java (use package-private or inner classes instead)

13. CONSTRUCTOR & DESTRUCTOR

Constructor

- **Definition:** Special member function called automatically when object is created
- **Rules:** Same name as class; no return type (not even void); can be overloaded; NOT virtual (usually)

Type	Description
Default Constructor	No parameters; provided by compiler if none defined
Parameterized Constructor	Takes arguments to initialize members with specific values
Copy Constructor	Creates new object as copy of existing object; C++: <code>ClassName(const ClassName &obj)</code>
Constructor Overloading	Multiple constructors with different parameter lists

Destructor

- **Definition:** Special function called automatically when object goes out of scope or is deleted
- **Rules:** Same name as class with ~ prefix; no parameters; no return type; cannot be overloaded; only ONE per class
- **Syntax (C++):** `~ClassName() { // cleanup code }`
- **Java:** `finalize()` method (deprecated in Java 9+; use try-with-resources)
- **Virtual destructor:** MUST use in base class if deleting derived class via base pointer
- **Order:** Derived destructor → Base destructor (reverse of construction)

Constructor Chaining

- Java: `this()` calls another constructor in same class; `super()` calls parent constructor
- C++: Use member initializer list — `Derived(): Base(), data(0) { }`

14. OTHER IMPORTANT CONCEPTS

this Keyword

- **Refers to:** Current object instance
- **Uses:** Resolve naming conflict (`this.name = name`), pass current object, call constructor (`this()`)
- **Not in static:** Cannot use 'this' in static method

super Keyword (Java)

- **Refers to:** Parent class object
- **Uses:** Call parent constructor (`super()`), access parent method (`super.method()`), access parent field (`super.field`)
- **Must be first:** `super()` must be first statement in child constructor

Interface

- **Definition:** 100% abstract type (pre-Java 8); specifies WHAT to do, not HOW
- **Keywords:** `interface` to declare, `implements` to use
- **Variables:** All are `public static final` by default
- **Methods:** All `public abstract` by default (Java 8 added default + static methods)
- **Multiple:** A class can implement MULTIPLE interfaces (solves multiple inheritance)
- **Extends:** Interface can extend multiple interfaces

Method Hiding vs Overriding

Feature	Method Overriding	Method Hiding
Applies to	Instance methods	Static methods
Resolved at	Runtime (dynamic)	Compile time (static)
Polymorphism	Yes (runtime)	No

Operator Overloading (C++)

- **Definition:** Giving special meaning to operators for user-defined types
- **Syntax:** `returnType operator+(ClassName obj) { ... }`
- **Cannot overload:** `::` (scope), `.*`, `?:` (ternary), `sizeof`, `typeid`
- **Java:** Does NOT support operator overloading (except `+` for String internally)

Object Slicing (C++)

- **Definition:** When derived class object assigned to base class variable, derived-specific data is SLICED OFF
- **Fix:** Use pointers or references to base class

Message Passing

- **Definition:** Process of one object communicating with another by calling methods
- **Key to OOP:** Objects interact via messages (method calls), not direct data access

Association, Aggregation, Composition

Concept	Description	Lifetime
Association	General relationship between objects (HAS-A loose)	Independent
Aggregation	Weak HAS-A; part can exist without whole	Independent (PART can survive)
Composition	Strong HAS-A; part cannot exist without whole	Dependent (part dies with whole)

15. EXAM QUICK-FIRE FACTS

- **Java does NOT support:** Multiple inheritance (classes), operator overloading, friend functions, destructor, pointers
- **C++ supports but Java does NOT:** Multiple inheritance, templates (Java has generics), friend, operator overloading, destructor (~)
- **Pure OOP languages:** Smalltalk, Ruby, Eiffel — everything is an object
- **Abstract class with 0 concrete methods =** Same as interface (in concept)
- **Virtual function table (vtable):** Created per class, not per object; vptr created per object
- **Constructor is NOT virtual** (in general); Destructor SHOULD be virtual in base class
- **final class examples:** String, Integer, Math, StringBuffer, StringBuilder in Java
- **static variable:** Only one copy shared by all objects — like a class-level global
- **Encapsulation → Data Hiding | Abstraction → Implementation Hiding**
- **Inheritance types in Java:** Single, Multilevel, Hierarchical (NOT Multiple, NOT Hybrid via classes)

- **Interface vs Abstract:** Use Interface for full abstraction + multiple inheritance; Abstract class when sharing code among related classes
- **Method Signature:** Method name + parameter list (return type NOT part of signature)
- **Covariant Return Type:** Overriding method can return subtype of parent method's return type (Java 5+)

BEST OF LUCK FOR UGC NET!

Revise, attempt MCQs, trust yourself.